UrbanEye AI-Driven Smart Complaint & Civic Issue Resolution System

(ES-452: Major Project - Dissertation)

submitted in partial fulfillment of the requirement
for the award of the degree of

Bachelor of Technology
in
Information Technology

Submitted by

ALOK RANJAN
15276803122

Under the supervision of

MRS. JASLEEN KAUR
ASSISTANT PROFESSOR

Information Technology
Guru Tegh Bahadur Institute of Technology
G-8 Area, Rajouri Garden, New Delhi - 110064

May/June 2026

DECLARATION

This is to certify that the material embodied in this Major Project - Dissertation titled “UrbanEye – AI-Driven Smart Complaint & Civic Issue Resolution System” being submitted in the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology is based on my original work. It is further certified that this Major Project - Dissertation work has not been submitted in full or in part to this university or any other university for the award of any other degree or diploma. My indebtedness to other works has been duly acknowledged at the relevant places.

(Alok Ranjan)
15276803122

CERTIFICATE

This is to certify that the work embodied in this Major Project - Dissertation titled “UrbanEye – AI-Driven Smart Complaint & Civic Issue Resolution System” being submitted in the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology, is original and has been carried out by ALOK RANJAN (Enrollment No. 15276803122) under my supervision and guidance.

It is further certified that this Major Project - Dissertation work has not been submitted in full or in part to this university or any other university for the award of any other degree or diploma to the best of my knowledge and belief.

(Mrs. Jasleen Kaur)
Assistant Professor

(Dr. Savneet Kaur)
HOD
Guru Tegh Bahadur Institute of Technology

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who have supported and guided me throughout the successful completion of my major project titled “UrbanEye: AI-Driven Smart Complaint & Civic Issue Resolution System.”

I am deeply indebted to my faculty supervisor and mentor, Mrs. Jasleen Kaur, for her continuous guidance, invaluable suggestions, and constant encouragement. Her expertise, patience, and insightful feedback played a vital role in shaping this project and strengthening both the technical and academic quality of the work.

I would also like to express my sincere thanks to the Head of the Department, Dr. Savneet Kaur, Department of Information Technology, for her support, motivation, and for providing the academic environment required to complete this project successfully.

Furthermore, I extend my gratitude to all the faculty members of the Information Technology Department at Guru Tegh Bahadur Institute of Technology for their cooperation and support during the course of this work.

Lastly, I would like to thank my friends and peers for their encouragement, collaboration, and moral support, which helped me stay motivated and overcome challenges throughout the development of this project.

ABSTRACT

UrbanEye is a full-stack civic complaint and issue-resolution platform developed to improve the way public infrastructure problems are reported, analyzed, and tracked. In many existing complaint systems, citizens are able to submit issues, but they often receive little clarity about how the complaint is categorized, where it is routed, and whether any real action is being taken. This project addresses that limitation by transforming complaint submission from a one-time reporting activity into a structured and trackable workflow.

The platform integrates a web application, a mobile application, a backend API, and a FastAPI-based AI service to support end-to-end complaint handling. A user can report an issue by entering a title, description, location, and optional image data. The AI layer then enriches the complaint by identifying the likely category, estimating priority, suggesting the responsible department, generating a recommended action, and drafting a concise X-ready public update. This makes complaint records more meaningful and useful for both citizens and administrative authorities.

UrbanEye also provides dashboard-based tracking, complaint status visibility, and location-oriented monitoring features. The mobile application is designed to help residents easily check whether a complaint is pending, in progress, or resolved, while the web application focuses on broader operational visibility through summaries, complaint views, and administrative monitoring tools. The system also supports fallback behavior for local development and demonstration, making it resilient even when some external services are unavailable.

Overall, UrbanEye demonstrates how AI can be applied in a practical civic-tech environment to improve transparency, accountability, and structured decision-making in public grievance systems. The project serves as a scalable and extensible foundation for future enhancements such as direct social media publishing, image-based issue detection, multilingual complaint handling, and deeper integration with smart city governance workflows.

LIST OF FIGURES

LIST OF TABLES

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

Executive Summary

UrbanEye is a full-stack civic complaint and issue-resolution platform designed to improve the way public issues are reported, analyzed, and tracked. The project was developed to address a common problem in existing grievance systems: citizens can submit complaints, but they often do not know what happens after submission, which department is responsible, or whether any action has started. UrbanEye solves this by turning complaint registration into a structured, transparent, and trackable workflow.

The platform combines a web application, a mobile application, a backend API, and a FastAPI-based AI service into one integrated system. Instead of storing a complaint only as plain text, UrbanEye enriches it with structured information such as category, priority, likely department, suggested action, and an X-ready public update. This makes complaint records more useful for both citizens and administrators.

UrbanEye also brings together complaint reporting, AI-assisted triage, dashboard-based monitoring, status visibility, and local fallback support in one ecosystem. Built using modern technologies such as Next.js, React, TypeScript, Node.js, Express.js, Prisma, FastAPI, React Native, and Expo, the project demonstrates modular design, practical AI integration, and deployment readiness suitable for civic-tech applications.

Key Metrics:

• Codebase: multi-module full-stack project across web, mobile, backend, and AI service

• Frontend stack: Next.js, React, TypeScript, Tailwind CSS

• Mobile stack: React Native, Expo, AsyncStorage

• Backend stack: Node.js, Express.js, Prisma

• AI stack: FastAPI, Pydantic, optional LLM integration

• Core focus: complaint reporting, AI-assisted classification, dashboard tracking, and public-update drafting

What is UrbanEye?

UrbanEye is a comprehensive civic complaint and monitoring platform that transforms traditional complaint filing into an intelligent and trackable system. Unlike basic reporting portals that only capture complaint text and store it in a database, UrbanEye connects complaint submission, AI-assisted analysis, priority and department suggestion, dashboard-based tracking, status visibility for residents, location-aware complaint monitoring, and X-ready public communication support into one workflow.

The goal of the platform is not only to collect complaints, but also to make them easier to understand, manage, and follow up on.

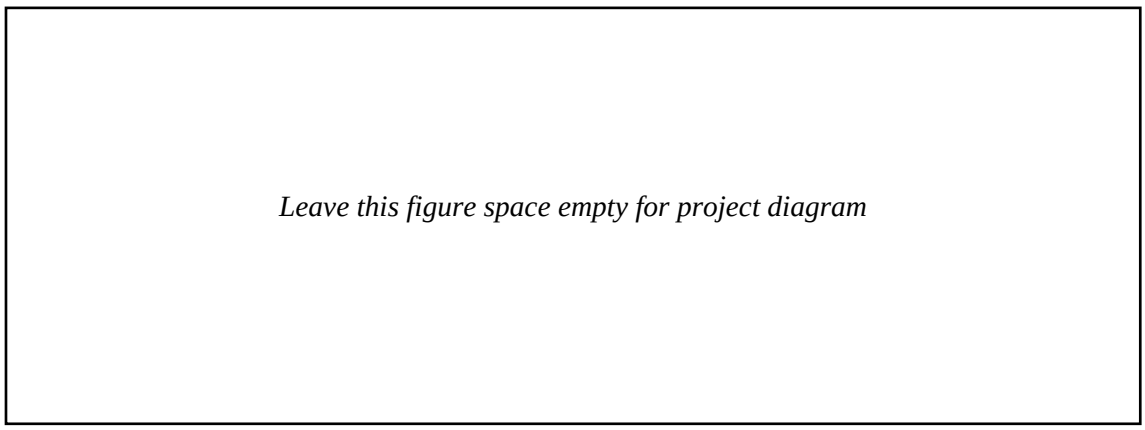

Figure 1.1: Domains Integration in UrbanEye

Primary Vision

"Create a unified, citizen-friendly complaint and civic issue resolution platform that improves public reporting through AI-assisted triage, structured tracking, dashboard visibility, and accountable digital workflows."

Table 1.1: Strategic Objectives

Objective	Description	Implementation
Complaint Reporting	Make civic issue reporting simple and accessible	Web form, mobile complaint submission flow
AI-Assisted Triage	Add structure to complaints automatically	Category, priority, department, and suggested action g
Status Transparency	Help citizens track complaint progress clearly	Pending, in-progress, and resolved complaint states
Administrative Visibility	Support monitoring and overview of complaints	Dashboard summaries, complaint cards, hotspot-orien
Public Communication	Prepare complaint-related public updates	X-ready social post drafting
Resilience	Keep the platform usable during service issues	Fallback logic, local persistence, mock-backed continu

Workflow 1: New User Complaint Journey

Step 1: Landing on Homepage

When the user opens the platform, they are presented with a civic complaint dashboard and entry points for reporting issues. The homepage introduces the purpose of the system and gives a quick view of complaint activity.

Step 2: Reporting a Civic Issue

The user creates a complaint by entering the title, description, and location of the issue. Depending on the interface, optional image and coordinate support may also be available.

Step 3: AI-Based Complaint Enrichment

Once the complaint is submitted, the backend sends the complaint text to the AI service. The AI layer analyzes the complaint and enriches it with structured fields such as complaint category, estimated priority, likely department, suggested action,

sentiment, and X-ready public update text.

Step 4: Complaint Storage and Routing Support

After analysis, the complaint is stored in the system with both original and AI-generated fields. This allows the complaint to be displayed consistently across web and mobile interfaces.

Step 5: Complaint Tracking

Once stored, the user can track the complaint in the dashboard. UrbanEye makes the status visible using simple complaint states: Pending, In Progress, and Resolved.

Step 6: Dashboard Monitoring and Follow-Up

On the web side, dashboards help provide an overview of complaint flow, summaries, and monitoring. On the mobile side, the focus is more personal and status-oriented, helping residents see what is solved and what still needs attention.

CHAPTER 2: PROBLEM STATEMENT

2.1: Problem Definition

Existing civic complaint and grievance systems often fail to provide a clear, structured, and transparent process for reporting and tracking public issues. Citizens may submit complaints about problems such as garbage overflow, water leakage, broken streetlights, potholes, or drainage blockage, but they often do not know which department is responsible, how urgent the issue is, or whether any action has been taken.

At the same time, authorities frequently receive complaints in unstructured form, making classification, prioritization, and monitoring more difficult. This creates inefficiency on both sides of the complaint process.

In many cities, complaint reporting is fragmented across websites, apps, phone calls, and social media channels. This makes the process repetitive for citizens and analytically weak for administrators.

Problem 1: Fragmentation of Complaint Reporting and Tracking Tools

In the current civic complaint environment, citizens often depend on multiple disconnected channels to report, follow up, and escalate public issues. This creates a fragmented and inefficient experience for both citizens and authorities.

• Online complaint portals often require long forms and provide limited post-submission clarity.

• Mobile apps may allow issue reporting but only basic status visibility.

• Phone calls and helplines force users to repeat information repeatedly.

• Social media posts create visibility but are usually disconnected from formal complaint records.

• Manual notes and screenshots become the user’s only tracking mechanism in many cases.

Problem 2: Disconnected Complaint Reporting from Resolution Visibility

In many systems, complaint submission and complaint tracking exist as separate experiences. A citizen may be able to file a complaint, but the later stages such as classification, routing, action, and resolution are often unclear or poorly connected.

After filing the complaint, the user may repeatedly check status, contact departments manually, or escalate through social media. This weakens trust and makes the complaint process feel incomplete.

Table 2.1: Impact of Existing Complaint Workflow

Issue	Impact on Citizen	Impact on Administration
Fragmented channels	Confusion and repetition	Duplicate and inconsistent records
Weak classification	No clarity on urgency or responsibility	Manual routing effort increases
Poor status visibility	Low trust in complaint system	Higher follow-up overhead
No structured dashboards	No meaningful progress feedback	Reduced complaint analytics quality

2.2: Objectives

• To create an integrated complaint management platform that joins reporting, AI-assisted triage, storage, and tracking.

• To provide civic-tech solutions focused on public issue reporting rather than generic task handling.

• To improve status transparency for citizens through visible complaint states.

• To strengthen administrative visibility with summaries, breakdowns, and hotspot-oriented views.

• To support public communication readiness through X-ready social post drafting.

• To maintain resilience through fallback logic, local persistence, and modular architecture.

CHAPTER 3: ANALYSIS

3.1: Software Requirement Specifications (SRS)

The Software Requirement Specifications document defines what the UrbanEye system must do, how it must perform, and what constraints it must satisfy.

3.1.1: Functional Requirements of the Project

Functional requirements specify what the system must do. UrbanEye is centered on complaint submission, complaint analysis, complaint retrieval, dashboard tracking, administrative visibility, X account preference support, and resilient fallback operation.

Requirement ID	Title	Priority	Description
FR-1.1	Submit a Civic Complaint	HIGH	The system must allow users to submit civic complaints throug
FR-1.2	Retrieve Complaint List	HIGH	The system must fetch and display complaint records in structu
FR-2.1	Analyze Complaint Content	HIGH	The AI service must enrich complaints with structured civic m
FR-2.2	Generate X-Ready Public Update	MEDIUM	The system must draft concise public communication text from
FR-3.1	Display Complaint Tracking Dashboard	HIGH	The system must show totals, status, and complaint progress in
FR-3.2	Display Administrative Overview	HIGH	The system must provide complaint summaries for monitoring
FR-4.1	Save X Account Preference	MEDIUM	The system must allow users to save a preferred X handle for l
FR-5.1	Support Fallback Operation	HIGH	The system must remain usable even when some services are u

3.1.2: Non-functional Requirements of the Project

Non-functional requirements specify how the system must perform, including quality attributes, constraints, and operational characteristics.

Category	Requirement Focus
Performance	Fast complaint submission, dashboard refresh, and AI-assisted complaint analysis
Throughput & Scalability	Ability to support rising complaint volume and concurrent access
Usability	Simple, clear, and readable complaint workflow for citizens
Accessibility	Inclusive design aligned with WCAG-oriented principles
Reliability	Fallback behavior and reduced failure impact
Maintainability	Modular structure across web, backend, mobile, and AI service layers

3.2: Feasibility Study of the Project

Technical Feasibility: UrbanEye is technically feasible because it uses a practical and well-supported stack including Next.js, React, Express, Prisma, FastAPI, React Native, and Expo. Each layer can be developed and tested independently while still contributing to one integrated platform.

Economic Feasibility: The prototype is economically feasible because it can be developed using open-source frameworks, local development workflows, and

student-friendly deployment options. Early-stage operation does not require enterprise-only infrastructure.

Operational Feasibility: The workflow matches real user expectations. Residents can report issues in simple language, and administrators receive more structured complaint records. This makes the system usable on both ends of the complaint process.

Schedule Feasibility: The project is feasible within a major-project timeline because complaint reporting, AI enrichment, dashboard visibility, and fallback behavior can be built incrementally and demonstrated progressively.

3.3: Tools / Technologies / Platform used

Table 3.3.1: Frontend Layer	Technology	Purpose
Web Frontend	Next.js, React, TypeScript, Tailwind CSS	Citizen and admin dashboards, forms, complaint cards, m
Mobile Frontend	React Native, Expo, AsyncStorage	Complaint reporting, tracking, local persistence, and mob
Shared UI Logic	Component-based design	Reusable interface building blocks

Table 3.3.2: Backend Layer	Technology	Purpose
API Layer	Node.js, Express.js	Complaint, admin, auth, and map route handling
Persistence	Prisma, PostgreSQL-ready schema, mock store	Complaint and user data management
Support Services	Cache service, validators, middleware	Resilience, validation, and response normalization

Table 3.3.3: External Services & APIs	Technology	Purpose
AI Service	FastAPI, Pydantic	Complaint analysis and X-ready social post generation
Mobile Local Storage	AsyncStorage	Local complaint continuity
Map Support	Project map service	Hotspot and location-oriented complaint visibility

Table 3.3.4: Deployment & Hosting	Technology	Purpose
Web Runtime	Managed web host / local development	Run web application
Backend Runtime	Node service host	Run complaint API
AI Runtime	Python service host	Run complaint analysis service

Table 3.3.5: Development Tools	Tool	Purpose
Version Control	Git, GitHub	Source control and collaboration
Editor	VS Code	Development environment
Testing Support	Syntax checks, Expo export, API verification	Stability validation

3.4: Use Case Diagrams / Data Flow Diagrams

Leave this use case diagram space empty

Figure 3.4.1: Use Case Diagram of UrbanEye

A use case diagram shows how the citizen user and the administrative user interact with UrbanEye. The citizen reports complaints, views complaint status, opens complaint details, and manages profile preferences. The administrative user reviews complaint totals, category distributions, and hotspot-oriented summaries.

Leave this Level 0 DFD space empty

Figure 3.4.2: Level 0 Data Flow Diagram (DFD)

The Level 0 DFD shows complaint data moving from web or mobile interfaces to the backend API, then to the AI service and persistence layer, before being returned to dashboards, detail views, and administrative screens.

Leave this Level 1 DFD space empty

Figure 3.4.3: Level 1 Data Flow Diagram (DFD)

The Level 1 DFD expands the complaint lifecycle into validation, analysis, storage, retrieval, and monitoring stages. It makes the complaint object the central information unit of the platform.

CHAPTER 4: DESIGN AND ARCHITECTURE

4.1: Structure Chart / Work Breakdown Structure

Leave this WBS space empty

Figure 4.1.1: Work Breakdown Structure (WBS)

A WBS decomposes UrbanEye into manageable implementation units showing hierarchy and dependencies across the web frontend, mobile app, backend API, AI service, and documentation/testing activities.

4.2: Explanation of Modules

The web application acts as the dashboard-oriented face of UrbanEye. It supports landing content, report panels, complaint cards, complaint detail access, map visibility, profile settings, and administrative summaries.

The mobile application is designed around ease of complaint reporting and personal complaint tracking. It emphasizes quick status interpretation, solved versus open progress, and local continuity.

The backend API coordinates complaint intake, validation, analysis calls, persistence, retrieval, caching, and administrative summaries. It acts as the operational core of the project.

The AI service is intentionally separated as a FastAPI microservice so that complaint analysis can evolve independently. It currently focuses on deterministic civic issue interpretation and X-ready public update generation.

The persistence layer combines database-ready structure with fallback continuity. When full persistence is not available, mock and local storage keep the complaint journey demonstrable and visible.

Leave this system architecture space empty

Figure 4.2.1: System Architecture Diagram

4.3: Flow Chart / Activity Diagram

Leave this activity diagram space empty

Figure 4.3.1: Activity Diagram: Complaint Submission Flow

The complaint submission flow begins when the resident enters complaint details, continues through validation and AI enrichment, and ends with structured storage and dashboard visibility.

4.4: ER Diagram / Class Diagram

Leave this ER / class diagram space empty

Figure 4.4.1: ER Diagram / Class Diagram

The core data model of UrbanEye centers on the complaint entity, which stores original complaint fields as well as AI-generated fields such as category, priority, department,

suggested action, and social post text. User association, location attributes, and status information complete the model.

CHAPTER 5: IMPLEMENTATION

5.1: Screenshots

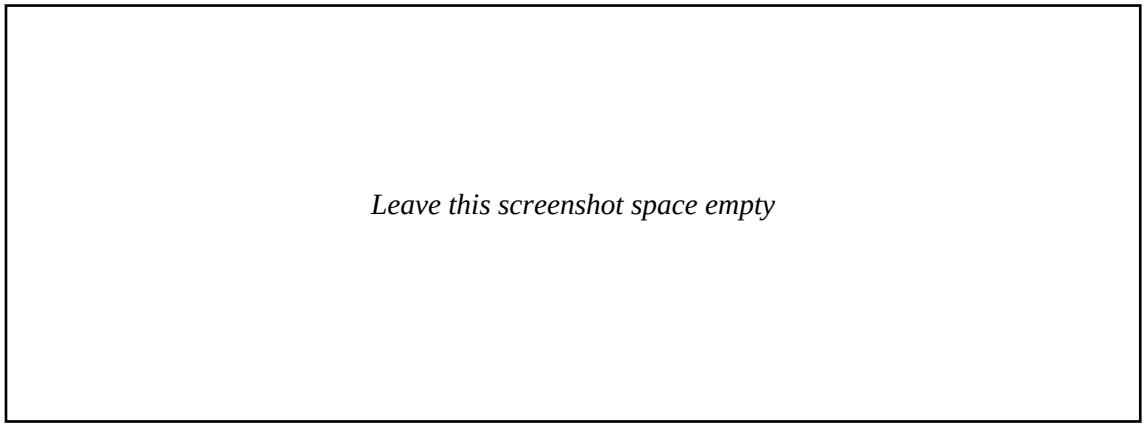

Figure 5.1.1: Home Page

The home page introduces UrbanEye as a citizen-friendly civic-tech system and provides complaint entry points, dashboard context, and system framing.

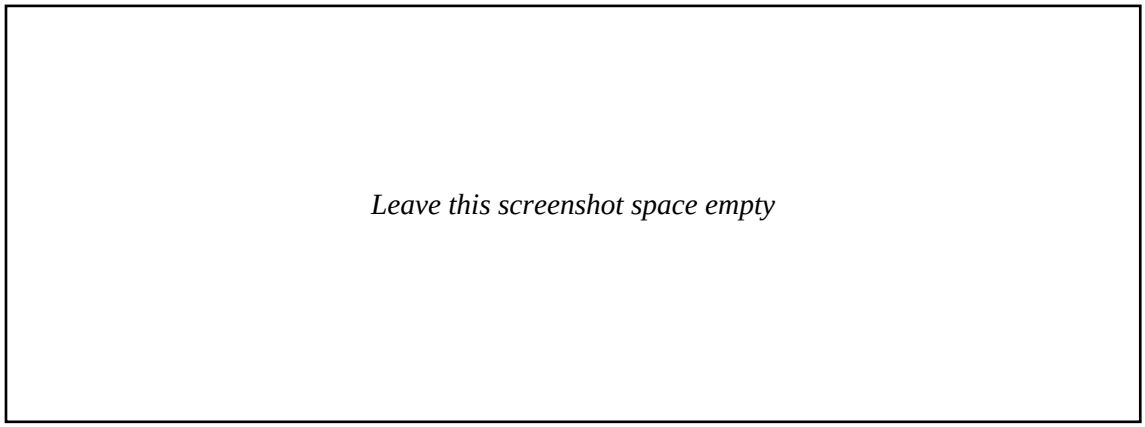

Figure 5.1.2: Dashboard Page

The dashboard page summarizes complaint flow, totals, status information, and operational overview components for users and administrators.

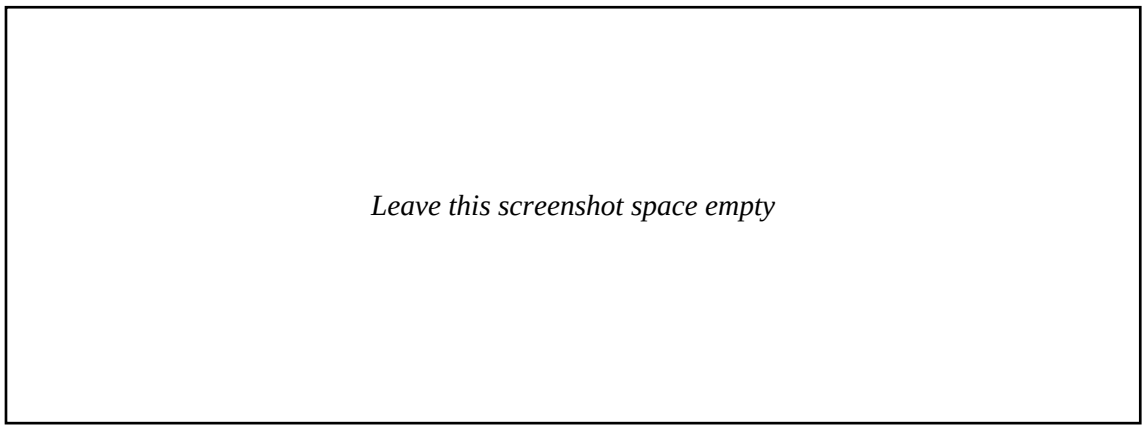

Figure 5.1.3: Profile Page

The profile page stores the preferred X account handle and keeps personal settings lightweight and accessible.

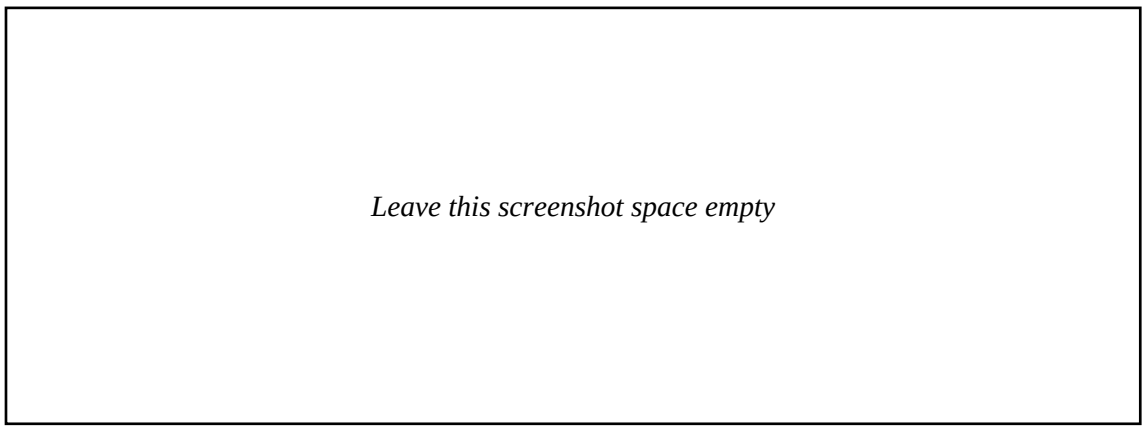

Figure 5.1.4: Complaint Detail Page

The complaint detail page shows original complaint content together with category, priority, department, suggested action, and social post drafting support.

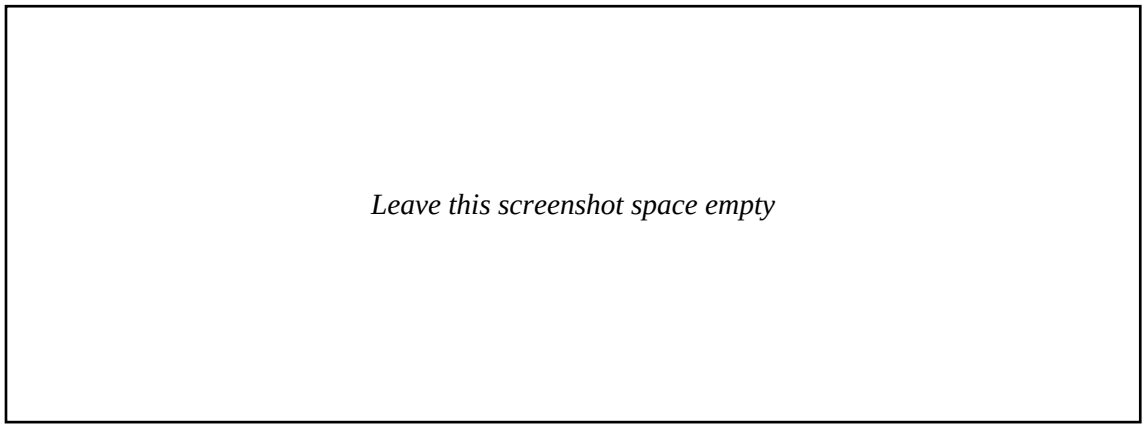

Figure 5.1.5: Administrative Overview Page

The administrative overview page displays complaint distributions, status totals, and map-oriented or hotspot-oriented insights.

5.2: Source Code of some modules

Selected source code modules are included in the appendix so that the report remains readable while still demonstrating the real project implementation.

CHAPTER 6: TESTING

Testing in UrbanEye focused on the full complaint journey rather than isolated functions alone. The most important question was whether a complaint could move from user input to structured output and then reappear reliably in tracking views.

Functional testing checked complaint submission, complaint retrieval, AI classification behavior, dashboard visibility, and mobile tracking features. Performance-oriented testing examined response behavior under repeated local use, while resilience testing examined fallback continuity when ideal service conditions were not available.

Test Case	Action	Expected Result	Status
Complaint Submission	Submit valid complaint payload	Complaint is created with AI-enriched metadata	Passed
Complaint Retrieval	Fetch all complaints	Normalized complaint records returned	Passed
Complaint Detail	Open one complaint	Structured complaint view displayed	Passed
AI Analysis	Submit roads / sanitation / water complaint text	Reasonable category and department mapping	Passed
Mobile Local Persistence	Create complaint while backend path fails	Complaint remains visible locally	Passed
Administrative Overview	Open overview dashboard	Complaint summaries are displayed	Passed

CHAPTER 7: SUMMARY & CONCLUSIONS

Project Summary

UrbanEye is a full-stack civic complaint and issue-resolution platform created to make public problem reporting easier, clearer, and more trackable. The project combines a web interface, a mobile interface, a backend API, and a FastAPI-based AI service in one unified workflow.

Its strongest contribution is the transformation of complaint submission into a visible lifecycle. Instead of stopping at complaint intake, the platform enriches complaints, persists them, and makes their progress visible through dashboards and detail views.

Conclusion

UrbanEye demonstrates that AI can support public grievance systems when it is embedded inside a structured application flow. The project improves transparency, complaint clarity, and readiness for accountable communication.

For a major project, UrbanEye is technically credible and socially relevant because it combines real software engineering, practical AI usage, multi-platform design, and a meaningful public-service problem domain in one system.

CHAPTER 8: LIMITATION OF THE PROJECT & FUTURE WORK

Limitations

• The present AI workflow is mainly text-driven and does not yet provide full image-based issue detection in the active complaint path.

• The platform generates X-ready public updates, but direct supervised live posting is not yet completed.

• The current project is optimized for academic demonstration rather than immediate city-wide production rollout.

• Map and hotspot functionality remain simpler than a full GIS-backed civic deployment.

• Multilingual complaint handling and large-scale policy governance remain future concerns.

Future Work & Roadmap

• Add supervised live X posting with approval and retry handling.

• Add stronger image-assisted civic issue understanding.

• Expand multilingual complaint analysis and broader dataset support.

• Introduce density-aware hotspot clustering and smarter priority assignment.

• Add richer administrative workflow states, notifications, and escalation history.

• Strengthen observability, privacy controls, and audit-friendly deployment behavior.

BIBLIOGRAPHY

Next.js Documentation, Vercel.

React Documentation, Meta.

Express.js Documentation.

FastAPI Documentation.

Prisma ORM Documentation.

Expo Documentation.

WCAG 2.1 Guidelines, W3C.

Research and civic-tech references relevant to complaint systems, dashboards, and responsible AI-assisted workflows.

APPENDIX

A. Important API Endpoints

Method	Endpoint	Purpose
POST	/auth/register	Register a user
POST	/auth/login	Authenticate a user
POST	/complaints	Create a complaint and attach AI analysis
GET	/complaints	Retrieve complaint list
GET	/complaints/:id	Retrieve single complaint
GET	/admin/dashboard	Administrative summary overview
GET	/admin/complaints	Administrative complaint list
GET	/map/hotspots	Complaint hotspot information
POST	/analyze	AI complaint analysis route

B. Stakeholder Analysis

Stakeholder	Role	Relevance
Citizen / Resident	Primary issue reporter	Defines usability and status transparency needs
Administrative User	Monitor and reviewer	Defines dashboard, summary, and category breakdown nee
System Maintainer	Technical maintainer	Needs modularity, logging, and service clarity
Future Field Team	Possible operational assignee	Motivates richer future workflow states

C. Extended Code Listings

Backend Application Bootstrap

File: backend/src/app.js

```
import express from "express";
import cors from "cors";
import cookieParser from "cookie-parser";

import authRoutes from "./routes/authRoutes.js";
import complaintRoutes from "./routes/complaintRoutes.js";
import adminRoutes from "./routes/adminRoutes.js";
import mapRoutes from "./routes/mapRoutes.js";
import userRoutes from "./routes/userRoutes.js";
import { isRedisReady } from "./config/redisConfig.js";

const app = express();

app.use(
  cors({
    origin: true,
    credentials: true,
  }),
);
app.use(express.json());
app.use(cookieParser());

app.get("/", (req, res) => {
  res.json({
    ok: true,
    name: "UrbanEye API",
    message: "API running",
  });
});

app.get("/health", (req, res) => {
  res.json({
    ok: true,
    redis: isRedisReady() ? "connected" : "unavailable",
  });
});

app.use("/api/auth", authRoutes);
app.use("/api/complaints", complaintRoutes);
app.use("/api/admin", adminRoutes);
app.use("/api/map", mapRoutes);
app.use("/api/users", userRoutes);

app.use((err, req, res, next) => {
  console.error(err);
  res.status(err.statusCode || 500).json({
    message: err.message || "Something went wrong",
  });
});

export default app;
```

Backend Server Entry

File: backend/src/server.js

```
import "dotenv/config";

import app from "./app.js";
import { connectRedis, disconnectRedis } from "./config/redisConfig.js";

const PORT = Number(process.env.PORT) || 5000;

await connectRedis();

const server = app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});

async function shutdown() {
  server.close(async () => {
    await disconnectRedis();
    process.exit(0);
  });
}

process.on("SIGINT", shutdown);
process.on("SIGTERM", shutdown);
```

Complaint Controller

File: backend/src/controllers/complaintController.js

```
import prisma from "../utils/prisma.js";
import {
  addComplaint,
  getComplaintById,
  getRawUsers,
  listComplaints,
} from "../data/mockStore.js";
import { analyzeComplaint } from "../services/aiService.js";
import {
  deleteCacheKeys,
  getCachedJson,
  setCachedJson,
} from "../services/cacheService.js";

const COMPLAINTS_CACHE_KEY = "complaints:all";
const COMPLAINT_CACHE_TTL_SECONDS = Number(process.env.COMPLAINT_CACHE_TTL_SECONDS) ||
     60;

function complaintCacheKey(id) {
  return `complaints:${id}`;
}

function normalizeComplaintShape(complaint) {
  return {
    ...complaint,
    category: complaint.category || "General",
    priority: complaint.priority || "MEDIUM",
    department: complaint.department || "Civic Response Cell",
    sentiment: complaint.sentiment || "concerned",
    confidence: complaint.confidence ?? 0.72,
    suggestedAction: complaint.suggestedAction || complaint.suggested_action || "",
    socialPost: complaint.socialPost || complaint.social_post || "",
    image: complaint.image || "",
    latitude: Number(complaint.latitude) || 25.5941,
    longitude: Number(complaint.longitude) || 85.1376,
    upvotes: Number(complaint.upvotes) || 0,
  };
}

export const createComplaint = async (req, res, next) => {
  try {
    const { title, description, location, image, latitude, longitude } = req.body;

    if (!title || !description || !location) {
      return res.status(400).json({ message: "Title, description, and location are
    required." });
    }

    const analysis = await analyzeComplaint({ title, description, location });

    if (prisma && process.env.DATABASE_URL) {
      const firstUser = await prisma.user.findFirst();

      if (!firstUser) {
        return res.status(400).json({ message: "Create a user before filing a
    complaint." });
      }

      const complaint = await prisma.complaint.create({
        data: {
          title: title.trim(),
          description: description.trim(),
          location: location.trim(),
          category: analysis.category,
          priority: analysis.priority,
          department: analysis.department,
          sentiment: analysis.sentiment,
          confidence: analysis.confidence,
          suggestedAction: analysis.suggestedAction,
```

```
          socialPost: analysis.socialPost,
          image: image || "",
          latitude: Number(latitude) || 25.5941,
          longitude: Number(longitude) || 85.1376,
          upvotes: 1,
          userId: firstUser.id,
        },
        include: {
          user: true,
        },
      });

      await deleteCacheKeys([COMPLAINTS_CACHE_KEY]);

      return res.status(201).json(
        normalizeComplaintShape(complaint),
      );
    }

    const defaultUser = getRawUsers()[0];
    const complaint = addComplaint({
      id: crypto.randomUUID(),
      title: title.trim(),
      description: description.trim(),
      location: location.trim(),
      ...analysis,
      status: "PENDING",
      image: image || "",
      latitude: Number(latitude) || 25.5941,
      longitude: Number(longitude) || 85.1376,
      upvotes: 1,
      userId: defaultUser.id,
      createdAt: new Date().toISOString(),
    });

    await deleteCacheKeys([COMPLAINTS_CACHE_KEY]);

    return res.status(201).json(normalizeComplaintShape(complaint));
  } catch (error) {
    return next(error);
  }
};

export const getComplaints = async (req, res, next) => {
  try {
    const cachedComplaints = await getCachedJson(COMPLAINTS_CACHE_KEY);
    if (cachedComplaints) {
      return res.json(cachedComplaints);
    }

    if (prisma && process.env.DATABASE_URL) {
      const complaints = await prisma.complaint.findMany({
        include: {
          user: true,
        },
        orderBy: {
          createdAt: "desc",
        },
      });

      const normalizedComplaints = complaints.map(normalizeComplaintShape);
      await setCachedJson(
        COMPLAINTS_CACHE_KEY,
        normalizedComplaints,
        COMPLAINT_CACHE_TTL_SECONDS,
      );

      return res.json(normalizedComplaints);
    }

    const complaints = listComplaints().map(normalizeComplaintShape);
    await setCachedJson(COMPLAINTS_CACHE_KEY, complaints,
    COMPLAINT_CACHE_TTL_SECONDS);
```

```
    return res.json(complaints);
  } catch (error) {
    return next(error);
  }
};

export const getComplaint = async (req, res, next) => {
  try {
    const cacheKey = complaintCacheKey(req.params.id);
    const cachedComplaint = await getCachedJson(cacheKey);
    if (cachedComplaint) {
      return res.json(cachedComplaint);
    }

    if (prisma && process.env.DATABASE_URL) {
      const complaint = await prisma.complaint.findUnique({
        where: {
          id: req.params.id,
        },
        include: {
          user: true,
        },
      });

      if (!complaint) {
        return res.status(404).json({ message: "Complaint not found." });
      }

      const normalizedComplaint = normalizeComplaintShape(complaint);
      await setCachedJson(cacheKey, normalizedComplaint, COMPLAINT_CACHE_TTL_SECONDS);

      return res.json(normalizedComplaint);
    }

    const complaint = getComplaintById(req.params.id);

    if (!complaint) {
      return res.status(404).json({ message: "Complaint not found." });
    }

    const normalizedComplaint = normalizeComplaintShape(complaint);
    await setCachedJson(cacheKey, normalizedComplaint, COMPLAINT_CACHE_TTL_SECONDS);

    return res.json(normalizedComplaint);
  } catch (error) {
    return next(error);
  }
};
```

Admin Controller

File: backend/src/controllers/adminController.js

```
import { getAnalytics, listComplaints, listUsers } from "../data/mockStore.js";

export function getAdminOverview(req, res) {
  res.json(getAnalytics());
}

export function getAdminComplaints(req, res) {
  res.json(listComplaints());
}

export function getAdminUsers(req, res) {
  res.json(listUsers());
}
```

Map Controller

File: backend/src/controllers/mapController.js

```
import { getMapHotspots } from "../data/mockStore.js";

export function getMapHotspotsController(req, res) {
  res.json(getMapHotspots());
}
```

Authentication Controller

File: backend/src/controllers/authcontroller.js

```
import bcrypt from "bcrypt";

import prisma from "../utils/prisma.js";
import { addUser, findUserByEmail } from "../data/mockStore.js";

function sanitizeUser(user) {
  if (!user) {
    return null;
  }

  const { password, ...safeUser } = user;
  return safeUser;
}

function buildUserPayload({ name, email, passwordHash, role = "CITIZEN" }) {
  return {
    id: crypto.randomUUID(),
    name,
    email,
    password: passwordHash,
    role,
    createdAt: new Date().toISOString(),
  };
}

export const register = async (req, res, next) => {
  try {
    const { name, email, password } = req.body;

    if (!name || !email || !password) {
      return res.status(400).json({ message: "Name, email, and password are required."
     });
    }

    const normalizedEmail = email.trim().toLowerCase();
    const passwordHash = await bcrypt.hash(password, 10);

    if (prisma && process.env.DATABASE_URL) {
      const existingUser = await prisma.user.findUnique({
        where: { email: normalizedEmail },
      });

      if (existingUser) {
        return res.status(409).json({ message: "User already exists." });
      }

      const user = await prisma.user.create({
        data: {
          name: name.trim(),
          email: normalizedEmail,
          password: passwordHash,
        },
      });

      return res.status(201).json({ user: sanitizeUser(user) });
    }

    const existingUser = findUserByEmail(normalizedEmail);
    if (existingUser) {
      return res.status(409).json({ message: "User already exists." });
    }

    const user = addUser(
      buildUserPayload({
        name: name.trim(),
        email: normalizedEmail,
        passwordHash,
      }),
    );
```

```
    return res.status(201).json({ user: sanitizeUser(user) });
  } catch (error) {
    return next(error);
  }
};

export const login = async (req, res, next) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({ message: "Email and password are required." });
    }

    const normalizedEmail = email.trim().toLowerCase();

    if (prisma && process.env.DATABASE_URL) {
      const user = await prisma.user.findUnique({
        where: { email: normalizedEmail },
      });

      if (!user) {
        return res.status(401).json({ message: "Invalid credentials." });
      }

      const isMatch = await bcrypt.compare(password, user.password);
      if (!isMatch) {
        return res.status(401).json({ message: "Invalid credentials." });
      }

      return res.json({ user: sanitizeUser(user) });
    }

    const user = findUserByEmail(normalizedEmail);
    if (!user) {
      return res.status(401).json({ message: "Invalid credentials." });
    }

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      return res.status(401).json({ message: "Invalid credentials." });
    }

    return res.json({ user: sanitizeUser(user) });
  } catch (error) {
    return next(error);
  }
};
```

User Controller

File: backend/src/controllers/userController.js

```
import { listUsers } from "../data/mockStore.js";

export function getCurrentUser(req, res) {
  res.json(listUsers()[0] || null);
}
```

Complaint Routes

File: backend/src/routes/complaintRoutes.js

```
import express from "express";
import {
  createComplaint,
  getComplaint,
  getComplaints,
} from "../controllers/complaintController.js";

const router = express.Router();

router.post("/", createComplaint);
router.get("/", getComplaints);
router.get("/:id", getComplaint);

export default router;
```

Admin Routes

File: backend/src/routes/adminRoutes.js

```
import { Router } from "express";
import {
  getAdminComplaints,
  getAdminOverview,
  getAdminUsers,
} from "../controllers/adminController.js";

const router = Router();

router.get("/overview", getAdminOverview);
router.get("/complaints", getAdminComplaints);
router.get("/users", getAdminUsers);

export default router;
```

Map Routes

File: backend/src/routes/mapRoutes.js

```
import { Router } from "express";
import { getMapHotspotsController } from "../controllers/mapController.js";

const router = Router();

router.get("/hotspots", getMapHotspotsController);

export default router;
```

Auth Routes

File: backend/src/routes/authRoutes.js

```
import express from "express";
import { register, login } from "../controllers/authController.js";

const router = express.Router();

router.post("/register", register);
router.post("/login", login);

export default router;
```

User Routes

File: backend/src/routes/userRoutes.js

```
import { Router } from "express";
import { getCurrentUser } from "../controllers/userController.js";

const router = Router();

router.get("/me", getCurrentUser);

export default router;
```

Backend AI Service Adapter

File: backend/src/services/aiService.js

```
const AI_SERVICE_URL = process.env.AI_SERVICE_URL?.replace(/\/$/, "") ||
    "http://localhost:8000";

const SOCIAL_HASHTAGS = {
  Sanitation: ["#CleanStreets", "#CivicAction"],
  Electricity: ["#StreetlightFix", "#CivicAction"],
  Water: ["#WaterAlert", "#CivicAction"],
  Roads: ["#RoadSafety", "#CivicAction"],
  General: ["#CityUpdate", "#CivicAction"],
};

function keywordMatch(text, keywords) {
  return keywords.some((keyword) => text.includes(keyword));
}

function compactText(text = "") {
  return text.replace(/\s+/g, " ").trim();
}

function trimToLength(text, limit) {
  const compact = compactText(text);
  if (compact.length <= limit) {
    return compact;
  }

  const trimmed = compact.slice(0, Math.max(limit - 1, 0)).replace(/[ ,.;:-]+$/, "");
  return `${trimmed}…`;
}

function buildSocialPost({ title = "", description = "", location = "", category =
    "General", priority = "MEDIUM", department = "Civic Response Cell" }) {
  const prefixByPriority = {
    CRITICAL: "Urgent civic alert:",
    HIGH: "High-priority civic alert:",
    MEDIUM: "Civic update:",
    LOW: "Civic update:",
  };
  const hashtags = (SOCIAL_HASHTAGS[category] || SOCIAL_HASHTAGS.General).join(" ");
  const issueSummary = trimToLength(description || title, 85);

  return trimToLength(
    `${prefixByPriority[priority] || prefixByPriority.MEDIUM} ${title} at ${location}.
     ${issueSummary} Assigned to ${department}. ${hashtags}`,
    280,
  );
}

function localAnalysis({ title = "", description = "", location = "" }) {
  const normalized = `${title} ${description} ${location}`.toLowerCase();

  let category = "General";
  let department = "Civic Response Cell";
  let priority = "MEDIUM";

  if (keywordMatch(normalized, ["garbage", "waste", "trash"])) {
    category = "Sanitation";
    department = "Sanitation Department";
    priority = "HIGH";
  } else if (keywordMatch(normalized, ["light", "electric", "power"])) {
    category = "Electricity";
    department = "Electricity Department";
  } else if (keywordMatch(normalized, ["water", "leak", "pipeline"])) {
    category = "Water";
    department = "Water Department";
    priority = "HIGH";
  } else if (keywordMatch(normalized, ["pothole", "road", "traffic", "drainage"])) {
    category = "Roads";
    department = "Road Department";
    priority = "HIGH";
```

```
  }

  if (keywordMatch(normalized, ["fire", "accident", "flood", "unsafe"])) {
    priority = "CRITICAL";
  }

  return {
    category,
    priority,
    department,
    sentiment: priority === "CRITICAL" ? "urgent" : "concerned",
    confidence: 0.72,
    suggestedAction: `Route this complaint to ${department} for field validation.`,
    socialPost: buildSocialPost({
      title,
      description,
      location,
      category,
      priority,
      department,
    }),
  };
}

export async function analyzeComplaint(payload) {
  try {
    const response = await fetch(`${AI_SERVICE_URL}/analyze`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(payload),
    });

    if (!response.ok) {
      throw new Error("AI service unavailable");
    }

    return await response.json();
  } catch (error) {
    return localAnalysis(payload);
  }
}
```

Backend Notification Service

File: backend/src/services/notificationService.js

Backend Cache Service

File: backend/src/services/cacheService.js

```
import { getRedisClient, isRedisReady } from "../config/redisConfig.js";

export async function getCachedJson(key) {
  if (!isRedisReady()) {
    return null;
  }

  try {
    const cachedValue = await getRedisClient().get(key);
    return cachedValue ? JSON.parse(cachedValue) : null;
  } catch (error) {
    console.warn(`Redis cache read failed for ${key}:`, error.message);
    return null;
  }
}

export async function setCachedJson(key, value, ttlSeconds = 60) {
  if (!isRedisReady()) {
    return;
  }

  try {
    await getRedisClient().set(key, JSON.stringify(value), {
      EX: ttlSeconds,
    });
  } catch (error) {
    console.warn(`Redis cache write failed for ${key}:`, error.message);
  }
}

export async function deleteCacheKeys(keys) {
  if (!isRedisReady() || keys.length === 0) {
    return;
  }

  try {
    await getRedisClient().del(keys);
  } catch (error) {
    console.warn("Redis cache delete failed:", error.message);
  }
}
```

Backend Map Service

File: backend/src/services/mapService.js

Backend Mock Store

File: backend/src/data/mockStore.js

```
const users = [
  {
    id: "seed-user-1",
    name: "Aarav Singh",
    email: "citizen@urbaneye.dev",
    password: "$2b$10$9cuvx0ExNkQWXdQ2n6A0n.6M3NSpQqVQ8jQJ4x6cPX0T3CB7wWQ1K",
    role: "CITIZEN",
    createdAt: "2026-03-18T08:20:00.000Z",
  },
  {
    id: "seed-user-2",
    name: "Meera Rao",
    email: "authority@urbaneye.dev",
    password: "$2b$10$9cuvx0ExNkQWXdQ2n6A0n.6M3NSpQqVQ8jQJ4x6cPX0T3CB7wWQ1K",
    role: "AUTHORITY",
    createdAt: "2026-03-18T09:00:00.000Z",
  },
  {
    id: "seed-user-3",
    name: "Sonal Verma",
    email: "admin@urbaneye.dev",
    password: "$2b$10$9cuvx0ExNkQWXdQ2n6A0n.6M3NSpQqVQ8jQJ4x6cPX0T3CB7wWQ1K",
    role: "ADMIN",
    createdAt: "2026-03-18T09:30:00.000Z",
  },
];

const complaints = [
  {
    id: "cmp-1001",
    title: "Overflowing roadside garbage",
    description: "Garbage bins near Sector 8 market have not been cleared for three
    days.",
    location: "Sector 8 Market, Patna",
    category: "Sanitation",
    priority: "HIGH",
    status: "IN_PROGRESS",
    department: "Sanitation Department",
    sentiment: "frustrated",
    confidence: 0.89,
    suggestedAction: "Dispatch sanitation pickup team within 12 hours.",
    socialPost: "Ticket escalated for sanitation crew update.",
    image: "",
    latitude: 25.6175,
    longitude: 85.1452,
    upvotes: 19,
    userId: "seed-user-1",
    createdAt: "2026-03-22T09:30:00.000Z",
  },
  {
    id: "cmp-1002",
    title: "Streetlight outage",
    description: "Two consecutive streetlights are out near the school crossing.",
    location: "Boring Road Crossing, Patna",
    category: "Electricity",
    priority: "MEDIUM",
    status: "PENDING",
    department: "Electricity Department",
    sentiment: "concerned",
    confidence: 0.78,
    suggestedAction: "Assign to night maintenance team for inspection.",
    socialPost: "",
    image: "",
    latitude: 25.6128,
    longitude: 85.1178,
    upvotes: 11,
    userId: "seed-user-1",
    createdAt: "2026-03-21T18:10:00.000Z",
  },
```

```
  {
    id: "cmp-1003",
    title: "Water leakage near bus stop",
    description: "Continuous water leakage is flooding the footpath near the bus
    stop.",
    location: "Kankarbagh Main Road, Patna",
    category: "Water",
    priority: "HIGH",
    status: "PENDING",
    department: "Water Department",
    sentiment: "urgent",
    confidence: 0.84,
    suggestedAction: "Inspect pipeline pressure and send repair crew.",
    socialPost: "",
    image: "",
    latitude: 25.5944,
    longitude: 85.1612,
    upvotes: 23,
    userId: "seed-user-1",
    createdAt: "2026-03-23T06:45:00.000Z",
  },
  {
    id: "cmp-1004",
    title: "Pothole causing traffic slowdown",
    description: "A deep pothole at the flyover approach is forcing bikes into
    traffic.",
    location: "Bailey Road Flyover, Patna",
    category: "Roads",
    priority: "HIGH",
    status: "RESOLVED",
    department: "Road Department",
    sentiment: "annoyed",
    confidence: 0.92,
    suggestedAction: "Mark area and schedule resurfacing crew.",
    socialPost: "Resolved after emergency patch work.",
    image: "",
    latitude: 25.6096,
    longitude: 85.1081,
    upvotes: 31,
    userId: "seed-user-1",
    createdAt: "2026-03-19T07:50:00.000Z",
  },
];

function decorateComplaint(complaint) {
  return {
    ...complaint,
    user: users.find((user) => user.id === complaint.userId) || null,
  };
}

export function listUsers() {
  return users.map(({ password, ...user }) => user);
}

export function getRawUsers() {
  return users;
}

export function findUserByEmail(email) {
  return users.find((user) => user.email.toLowerCase() === email.toLowerCase()) ||
    null;
}

export function addUser(user) {
  users.push(user);
  return user;
}

export function listComplaints() {
  return complaints
    .map(decorateComplaint)
    .sort((a, b) => new Date(b.createdAt) - new Date(a.createdAt));
```

```
}

export function getComplaintById(id) {
  const complaint = complaints.find((item) => item.id === id);
  return complaint ? decorateComplaint(complaint) : null;
}

export function addComplaint(complaint) {
  complaints.unshift(complaint);
  return decorateComplaint(complaint);
}

export function getAnalytics() {
  const items = listComplaints();
  const statusBreakdown = items.reduce((acc, item) => {
    acc[item.status] = (acc[item.status] || 0) + 1;
    return acc;
  }, {});
  const priorityBreakdown = items.reduce((acc, item) => {
    acc[item.priority] = (acc[item.priority] || 0) + 1;
    return acc;
  }, {});
  const departmentBreakdown = items.reduce((acc, item) => {
    acc[item.department] = (acc[item.department] || 0) + 1;
    return acc;
  }, {});

  return {
    totals: {
      complaints: items.length,
      pending: statusBreakdown.PENDING || 0,
      inProgress: statusBreakdown.IN_PROGRESS || 0,
      resolved: statusBreakdown.RESOLVED || 0,
      users: users.length,
    },
    statusBreakdown,
    priorityBreakdown,
    departmentBreakdown,
    recentComplaints: items.slice(0, 5),
  };
}

export function getMapHotspots() {
  return listComplaints().map((item) => ({
    id: item.id,
    title: item.title,
    location: item.location,
    latitude: item.latitude,
    longitude: item.longitude,
    category: item.category,
    priority: item.priority,
    status: item.status,
  }));
}
```

Complaint Validator

File: backend/src/validators/complaintValidators.js

Authentication Validator

File: backend/src/validators/authValidators.js

Authentication Middleware

File: backend/src/middlewares/authMiddleware.js

Error Middleware

File: backend/src/middlewares/errorMiddleware.js

Role Middleware

File: backend/src/middlewares/roleMiddleware.js

Multer Middleware

File: backend/src/middlewares/multerMiddleware.js

Backend Prisma Utility

File: backend/src/utils/prisma.js

```
let prisma = null;

if (process.env.DATABASE_URL) {
  try {
    const prismaPackage = await import("@prisma/client");
    const PrismaClient = prismaPackage.PrismaClient ||
    prismaPackage.default?.PrismaClient;

    prisma = PrismaClient ? new PrismaClient() : null;
  } catch (error) {
    prisma = null;
  }
}

export default prisma;
```

Prisma Schema

File: backend/prisma/schema.prisma

```
  generator client {
  provider = "prisma-client-js"
}

 datasource db {
  provider = "postgresql"
}

model User {
  id        String   @id @default(uuid())
  name      String
  email     String   @unique
  password  String
  role      Role     @default(CITIZEN)
  createdAt DateTime @default(now())

  complaints Complaint[]
}

model Complaint {
  id          String   @id @default(uuid())
  title       String
  description String
  location    String
  category    String   @default("General")
  priority    String   @default("MEDIUM")
  status      Status   @default(PENDING)
  department  String   @default("Civic Response Cell")
  sentiment   String   @default("concerned")
  confidence  Float    @default(0.0)
  suggestedAction String @default("")
  socialPost  String   @default("")
  image       String   @default("")
  latitude    Float    @default(25.5941)
  longitude   Float    @default(85.1376)
  upvotes     Int      @default(0)

  userId      String
  user        User     @relation(fields: [userId], references: [id])

  createdAt   DateTime @default(now())
}

enum Role {
  CITIZEN
  ADMIN
  AUTHORITY
}

enum Status {
  PENDING
  IN_PROGRESS
  RESOLVED
}
```

AI Service Main

File: ai_service/app/main.py

```
from fastapi import FastAPI
from pydantic import BaseModel, Field

from agents.social_agent import generate_social_post


class ComplaintPayload(BaseModel):
    title: str = Field(..., min_length=3)
    description: str = Field(..., min_length=5)
    location: str = Field(..., min_length=3)


class AnalysisResult(BaseModel):
    category: str
    priority: str
    department: str
    sentiment: str
    confidence: float
    suggestedAction: str
    socialPost: str


def keyword_match(text: str, keywords: list[str]) -> bool:
    return any(keyword in text for keyword in keywords)


def analyze_locally(payload: ComplaintPayload) -> AnalysisResult:
    text = f"{payload.title} {payload.description} {payload.location}".lower()

    category = "General"
    department = "Civic Response Cell"
    priority = "MEDIUM"
    sentiment = "concerned"

    if keyword_match(text, ["garbage", "waste", "trash"]):
        category = "Sanitation"
        department = "Sanitation Department"
        priority = "HIGH"
    elif keyword_match(text, ["light", "power", "electric"]):
        category = "Electricity"
        department = "Electricity Department"
    elif keyword_match(text, ["water", "leak", "pipeline"]):
        category = "Water"
        department = "Water Department"
        priority = "HIGH"
    elif keyword_match(text, ["pothole", "road", "traffic", "drainage"]):
        category = "Roads"
        department = "Road Department"
        priority = "HIGH"

    if keyword_match(text, ["fire", "accident", "flood", "unsafe"]):
        priority = "CRITICAL"
        sentiment = "urgent"

    action = f"Route this complaint to {department} and request field validation."
    social = generate_social_post(
        title=payload.title,
        description=payload.description,
        location=payload.location,
        category=category,
        priority=priority,
        department=department,
    )

    return AnalysisResult(
        category=category,
        priority=priority,
        department=department,
        sentiment=sentiment,
```

```
        confidence=0.74,
        suggestedAction=action,
        socialPost=social,
    )


app = FastAPI(
    title="UrbanEye AI Service",
    description="Complaint analysis and triage service for civic issue routing.",
    version="1.0.0",
)


@app.get("/health")
def health_check():
    return {"ok": True, "service": "urbaneye-ai"}


@app.post("/analyze", response_model=AnalysisResult)
def analyze_complaint(payload: ComplaintPayload):
    return analyze_locally(payload)
```

AI Social Agent

File: ai_service/agents/social_agent.py

```
from __future__ import annotations

import os
import re
from typing import Any

MAX_X_POST_LENGTH = 280

CATEGORY_HASHTAGS = {
    "Sanitation": ["#CleanStreets", "#CivicAction"],
    "Electricity": ["#StreetlightFix", "#CivicAction"],
    "Water": ["#WaterAlert", "#CivicAction"],
    "Roads": ["#RoadSafety", "#CivicAction"],
    "General": ["#CityUpdate", "#CivicAction"],
}


def _compact_text(value: str) -> str:
    return re.sub(r"\s+", " ", value).strip()


def _trim_to_length(value: str, limit: int) -> str:
    compact = _compact_text(value)
    if len(compact) <= limit:
        return compact
    trimmed = compact[: max(limit - 1, 0)].rstrip(" ,.;:-")
    return f"{trimmed}…" if trimmed else compact[:limit]


def _hashtags_for_category(category: str) -> str:
    tags = CATEGORY_HASHTAGS.get(category, CATEGORY_HASHTAGS["General"])
    return " ".join(tags)


def build_fallback_social_post(
    *,
    title: str,
    description: str,
    location: str,
    category: str,
    priority: str,
    department: str,
) -> str:
    priority_prefix = {
        "CRITICAL": "Urgent civic alert:",
        "HIGH": "High-priority civic alert:",
        "MEDIUM": "Civic update:",
        "LOW": "Civic update:",
    }.get(priority, "Civic update:")

    issue_summary = _trim_to_length(description or title, 85)
    hashtags = _hashtags_for_category(category)

    post = (
        f"{priority_prefix} {title} at {location}. "
        f"{issue_summary} Assigned to {department}. {hashtags}"
    )
    return _trim_to_length(post, MAX_X_POST_LENGTH)


def _llm_is_configured() -> bool:
    return bool(os.getenv("LLM_MODEL") and os.getenv("LLM_PROVIDER"))


def _extract_response_text(response: Any) -> str:
    if hasattr(response, "content"):
        return _compact_text(str(response.content))
    return _compact_text(str(response))
```

```
def generate_social_post(
    *,
    title: str,
    description: str,
    location: str,
    category: str,
    priority: str,
    department: str,
) -> str:
    fallback = build_fallback_social_post(
        title=title,
        description=description,
        location=location,
        category=category,
        priority=priority,
        department=department,
    )

    if not _llm_is_configured():
        return fallback

    try:
        from langchain_core.prompts import ChatPromptTemplate

        from config.llm import get_llm

        llm = get_llm()
        prompt = ChatPromptTemplate.from_messages(
            [
                (
                    "system",
                    (
                        "You write concise X posts for civic issue automation. "
                        f"Return exactly one post under {MAX_X_POST_LENGTH}
    characters. "
                        "Mention the issue and location, keep the tone factual and
    public-safe, "
                        "and end with two short relevant hashtags. No markdown
    fences."
                    ),
                ),
                (
                    "human",
                    (
                        "Title: {title}\n"
                        "Description: {description}\n"
                        "Location: {location}\n"
                        "Category: {category}\n"
                        "Priority: {priority}\n"
                        "Assigned department: {department}"
                    ),
                ),
            ]
        )
        chain = prompt | llm
        response = chain.invoke(
            {
                "title": title,
                "description": description,
                "location": location,
                "category": category,
                "priority": priority,
                "department": department,
            }
        )
        candidate = _trim_to_length(_extract_response_text(response),
    MAX_X_POST_LENGTH)
        return candidate or fallback
    except Exception:
        return fallback
```

AI Classification Agent

File: ai_service/agents/classification_agent.py

```
from config.llm import get_llm
from langchain_core.prompts import PromptTemplate


llm=get_llm()

prompt=PromptTemplate(
    input_variables=["complaint"],
    template="""
    you are an AI civic  complaint  analyzer.
    your task is to analyze the complaint and return structure JSON.

    complaint:{complaint}
    Categories:
- pothole
- garbage
- drainage
- streetlight
- water leak

Departments:
- Road Department
- Sanitation Department
- Water Department
- Electricity Department

Priority values:
low
medium
high

Return ONLY valid JSON in this format:

{{
 "category": "",
 "priority": "",
 "department": ""
}}

Priority values:
low
medium
high

"""
)

chain=prompt|llm

def analyze_complaint(complaint_text):
    response=chain.invoke({
        "complaint":complaint_text
        })
    return response.content
```

AI Priority Agent

File: ai_service/agents/priority_agent.py

```
from config.llm import get_llm
from langchain_core.prompts import PromptTemplate


llm=get_llm()
prompt=PromptTemplate(


    input_variable=["complaints"]
    template="""
    Determine priority of this civic complaint.

Complaint:
{complaint}

Rules:

HIGH:
- accidents
- hospital area
- dangerous roads

MEDIUM:
- drainage issues
- water leak

LOW:
- garbage collection

Return only one value:

low
medium
high
"""
)

chain=prompt|llm
def proiortize_complaint(complaints_text):

    response= chain.invoke({
       " complaints":{complaints_text}

    })
    return response.content
```

AI Routing Agent

File: ai_service/agents/routing_agent.py

AI Complaint Processor

File: ai_service/agents/complaint_processor.py

```
from config.llm import get_llm
from langchain_core.prompts import PromptTemplate

llm = get_llm()

prompt = PromptTemplate(
    input_variables=["complaint"],
    template="""
You are an AI civic complaint analyzer.

Analyze the complaint and return structured JSON.

Complaint:
{complaint}

Categories:
- pothole
- garbage
- drainage
- streetlight
- water leak

Departments:
- Road Department
- Sanitation Department
- Water Department
- Electricity Department

Return ONLY valid JSON in this format:

{
 "category": "",
 "priority": "",
 "department": ""
}

Priority values:
low
medium
high
"""
)

chain = prompt | llm


def process_complaint(complaint_text):

    response = chain.invoke({
        "complaint": complaint_text
    })

    return response.content
```

AI X Service

File: ai_service/services/x_service.py

AI RAG Service

File: ai_service/services/rag_service.py

Web Home Page

File: web_app/src/app/page.tsx

```
import Link from "next/link";

import Navbar from "../components/Navbar";
import ComplaintCard from "../components/Complaintcard";
import Sidebar from "../components/Sidebar";
import { demoComplaints } from "../lib/demoData";

const stats = [
  { label: "Open complaints", value: "128" },
  { label: "Resolved this week", value: "43" },
  { label: "Avg response time", value: "7.2h" },
];

export default function Home() {
  return (
    <div className="pb-14">
      <Navbar />
      <main className="mx-auto flex max-w-6xl flex-col gap-8 px-6 md:px-10">
        <section className="section-grid items-start">
          <div className="glass-card rounded-[40px] p-8 md:p-10">
            <p className="text-sm font-semibold uppercase tracking-[0.28em]
    text-[var(--accent-dark)]">
              Civic issue resolution system
            </p>
            <h1 className="mt-4 max-w-3xl text-5xl font-semibold leading-tight">
              Report city problems, route them faster, and keep residents informed.
            </h1>
            <p className="mt-5 max-w-2xl text-lg leading-8 text-[var(--ink-muted)]">
              UrbanEye brings together citizen reporting, admin visibility, and AI-
    assisted triage
              in the same workflow without changing your current stack.
            </p>
            <div className="mt-8 flex flex-wrap gap-4">
              <Link
                href="/dashboard"
                className="rounded-full bg-[var(--foreground)] px-6 py-3 text-sm font-
    semibold text-white transition hover:bg-[var(--accent-dark)]"
              >
                Open Dashboard
              </Link>
              <Link
                href="/register"
                className="rounded-full border border-[var(--border)] bg-white/75 px-6
     py-3 text-sm font-semibold"
              >
                Create Citizen Account
              </Link>
            </div>
            <div className="mt-10 grid gap-4 md:grid-cols-3">
              {stats.map((stat) => (
                <div key={stat.label} className="rounded-[28px] border
    border-[var(--border)] bg-white/65 p-5">
                  <p className="text-sm text-[var(--ink-muted)]">{stat.label}</p>
                  <p className="mt-3 text-3xl font-semibold">{stat.value}</p>
                </div>
              ))}
            </div>
          </div>
          <Sidebar />
        </section>

        <section className="grid gap-5 md:grid-cols-3">
          {demoComplaints.map((complaint) => (
            <ComplaintCard key={complaint.id} complaint={complaint} />
          ))}
        </section>
      </main>
    </div>
  );
```

```
}
```

Web Dashboard Page

File: web_app/src/app/dashboard/page.tsx

```
import DashboardShell from "../../components/DashboardShell";
import Navbar from "../../components/Navbar";
import Sidebar from "../../components/Sidebar";

export default function DashboardPage() {
  return (
    <div className="pb-12">
      <Navbar />
      <main className="mx-auto flex max-w-6xl flex-col gap-8 px-6 md:px-10">
        <section className="section-grid">
          <div className="glass-card rounded-[36px] p-8">
            <p className="text-xs font-semibold uppercase tracking-[0.24em]
    text-[var(--ink-muted)]">
              Operations board
            </p>
            <h1 className="mt-3 text-4xl font-semibold">Today&apos;s complaint flow at
     a glance</h1>
            <p className="mt-4 max-w-2xl text-[var(--ink-muted)]">
              Track incoming complaints, identify hotspots, and keep response teams
    aligned from
              one place.
            </p>
          </div>
          <Sidebar />
        </section>
        <DashboardShell />
      </main>
    </div>
  );
}
```

Web Profile Page

File: web_app/src/app/profile/page.tsx

```
import Navbar from "../../components/Navbar";

export default function ProfilePage() {
  return (
    <div className="pb-12">
      <Navbar />
      <main className="mx-auto max-w-4xl px-6 md:px-10">
        <section className="glass-card rounded-[36px] p-8">
          <p className="text-xs font-semibold uppercase tracking-[0.24em]
    text-[var(--ink-muted)]">
            Citizen profile
          </p>
          <h1 className="mt-3 text-4xl font-semibold">Resident account snapshot</h1>
          <div className="mt-8 grid gap-4 md:grid-cols-2">
            <div className="rounded-[28px] border border-[var(--border)] bg-white/70
    p-5">
              <p className="text-sm text-[var(--ink-muted)]">Name</p>
              <p className="mt-2 text-xl font-semibold">Aarav Singh</p>
            </div>
            <div className="rounded-[28px] border border-[var(--border)] bg-white/70
    p-5">
              <p className="text-sm text-[var(--ink-muted)]">Email</p>
              <p className="mt-2 text-xl font-semibold">citizen@urbaneye.dev</p>
            </div>
          </div>
        </section>
      </main>
    </div>
  );
}
```

Web Map Page

File: web_app/src/app/map/page.tsx

```
import Navbar from "../../components/Navbar";
import MapComplaint from "../../components/MapComplaint";
import { demoComplaints } from "../../lib/demoData";

export default function MapPage() {
  return (
    <div className="pb-12">
      <Navbar />
      <main className="mx-auto max-w-6xl px-6 md:px-10">
        <MapComplaint complaints={demoComplaints} />
      </main>
    </div>
  );
}
```

Web Complaint Page

File: web_app/src/app/complaints/page.tsx

```
import Navbar from "../../components/Navbar";
import ReportPanel from "../../components/ReportPanel";
import DashboardShell from "../../components/DashboardShell";

export default function ComplaintsPage() {
  return (
    <div className="pb-12">
      <Navbar />
      <main className="mx-auto max-w-6xl px-6 md:px-10">
        <section className="grid gap-8">
          <div className="glass-card rounded-[36px] p-8">
            <p className="text-xs font-semibold uppercase tracking-[0.24em]
    text-[var(--ink-muted)]">
              Complaint registry
            </p>
            <h1 className="mt-3 text-4xl font-semibold">Citizen reporting and live
    triage</h1>
          </div>
          <ReportPanel />
          <DashboardShell />
        </section>
      </main>
    </div>
  );
}
```

Web Admin Dashboard

File: web_app/src/app/admin/dashboard/page.tsx

```
import Navbar from "../../../components/Navbar";
import Sidebar from "../../../components/Sidebar";
import DashboardShell from "../../../components/DashboardShell";

export default function AdminDashboardPage() {
  return (
    <div className="pb-12">
      <Navbar />
      <main className="mx-auto max-w-6xl px-6 md:px-10">
        <section className="section-grid">
          <div className="glass-card rounded-[36px] p-8">
            <p className="text-xs font-semibold uppercase tracking-[0.24em]
    text-[var(--ink-muted)]">
              Admin console
            </p>
            <h1 className="mt-3 text-4xl font-semibold">Command center for urban
    operations</h1>
            <p className="mt-4 max-w-2xl text-[var(--ink-muted)]">
              Review triaged complaints, monitor response metrics, and keep
    departments aligned.
            </p>
          </div>
          <Sidebar />
        </section>
        <DashboardShell />
      </main>
    </div>
  );
}
```

Web Admin Complaints

File: web_app/src/app/admin/complaints/page.tsx

```
import Navbar from "../../../components/Navbar";
import ComplaintCard from "../../../components/Complaintcard";
import { demoComplaints } from "../../../lib/demoData";

export default function AdminComplaintsPage() {
  return (
    <div className="pb-12">
      <Navbar />
      <main className="mx-auto max-w-6xl px-6 md:px-10">
        <section className="grid gap-5 md:grid-cols-2">
          {demoComplaints.map((complaint) => (
            <ComplaintCard key={complaint.id} complaint={complaint} />
          ))}
        </section>
      </main>
    </div>
  );
}
```

Web Admin Analytics

File: web_app/src/app/admin/analytics/page.tsx

```
import Navbar from "../../../components/Navbar";

const metrics = [
  { label: "Average first response", value: "7.2h" },
  { label: "Resolution rate", value: "81%" },
  { label: "High priority backlog", value: "14" },
];

export default function AdminAnalyticsPage() {
  return (
    <div className="pb-12">
      <Navbar />
      <main className="mx-auto max-w-5xl px-6 md:px-10">
        <section className="glass-card rounded-[36px] p-8">
          <h1 className="text-4xl font-semibold">Analytics</h1>
          <div className="mt-8 grid gap-4 md:grid-cols-3">
            {metrics.map((metric) => (
              <div key={metric.label} className="rounded-[28px] border
    border-[var(--border)] bg-white/70 p-5">
                <p className="text-sm text-[var(--ink-muted)]">{metric.label}</p>
                <p className="mt-3 text-3xl font-semibold">{metric.value}</p>
              </div>
            ))}
          </div>
        </section>
      </main>
    </div>
  );
}
```

Web Login Page

File: web_app/src/app/login/page.tsx

```
"use client";

import Link from "next/link";
import { useRouter } from "next/navigation";
import { useState } from "react";

import { loginUser } from "../../services/authService";

export default function LoginPage() {
  const router = useRouter();
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [submitting, setSubmitting] = useState(false);
  const [message, setMessage] = useState("");

  const handleSubmit = async (event: React.FormEvent<HTMLFormElement>) => {
    event.preventDefault();

    if (!email || !password) {
      setMessage("Enter both email and password to continue.");
      return;
    }

    setSubmitting(true);
    setMessage("");

    try {
      const response = await loginUser({ email, password });
      if (typeof window !== "undefined") {
        window.localStorage.setItem("urbaneye-user", JSON.stringify(response.user));
      }
      setMessage("Login successful. Opening dashboard...");
      router.push("/dashboard");
    } catch (error) {
      setMessage(error instanceof Error ? error.message : "Login failed.");
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <main className="mx-auto flex min-h-screen max-w-6xl items-center px-6 py-12
    md:px-10">
      <div className="section-grid w-full">
        <section className="glass-card rounded-[36px] p-8 md:p-10">
          <p className="text-xs font-semibold uppercase tracking-[0.24em]
    text-[var(--accent-dark)]">
            Citizen access
          </p>
          <h1 className="mt-4 text-4xl font-semibold">Welcome back to UrbanEye</h1>
          <p className="mt-4 max-w-xl text-[var(--ink-muted)]">
            Sign in to track complaints, review AI triage, and submit new civic issues
     from the
            same dashboard.
          </p>
          <div className="mt-6 rounded-[24px] border border-[var(--border)] bg-
    white/65 p-5 text-sm text-[var(--ink-muted)]">
            Demo account: <span className="font-semibold
    text-[var(--foreground)]">citizen@urbaneye.dev</span>
            {" / "}
            <span className="font-semibold text-[var(--foreground)]">secret123</span>
          </div>
        </section>
        <section className="glass-card rounded-[36px] p-8">
          <form className="space-y-4" onSubmit={handleSubmit}>
            <input
              className="w-full rounded-2xl border border-[var(--border)] bg-white/85
    px-4 py-3 outline-none"
              placeholder="Email"
```

```
            type="email"
            value={email}
            onChange={(event) => setEmail(event.target.value)}
          />
          <input
            className="w-full rounded-2xl border border-[var(--border)] bg-white/85
    px-4 py-3 outline-none"
            placeholder="Password"
            type="password"
            value={password}
            onChange={(event) => setPassword(event.target.value)}
          />
          <button
            className="w-full rounded-full bg-[var(--foreground)] px-5 py-3 text-
    white transition hover:bg-[var(--accent-dark)] disabled:opacity-60"
            disabled={submitting}
            type="submit"
          >
            {submitting ? "Signing in..." : "Login"}
          </button>
        </form>
        {message ? <p className="mt-4 text-sm text-[var(--ink-muted)]">{message}</p>
     : null}
        <p className="mt-5 text-sm text-[var(--ink-muted)]">
          Need an account?{" "}
          <Link href="/register" className="font-semibold text-[var(--foreground)]">
            Register here
          </Link>
        </p>
      </section>
    </div>
  </main>
 );
}
```

Web Register Page

File: web_app/src/app/register/page.tsx

```
"use client";

import Link from "next/link";
import { useRouter } from "next/navigation";
import { useState } from "react";

import { registerUser } from "../../services/authService";

export default function RegisterPage() {
  const router = useRouter();
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [submitting, setSubmitting] = useState(false);
  const [message, setMessage] = useState("");

  const handleSubmit = async (event: React.FormEvent<HTMLFormElement>) => {
    event.preventDefault();

    if (!name || !email || !password) {
      setMessage("Fill in name, email, and password to create your account.");
      return;
    }

    setSubmitting(true);
    setMessage("");

    try {
      const response = await registerUser({ name, email, password });
      if (typeof window !== "undefined") {
        window.localStorage.setItem("urbaneye-user", JSON.stringify(response.user));
      }
      setMessage("Account created. Opening dashboard...");
      router.push("/dashboard");
    } catch (error) {
      setMessage(error instanceof Error ? error.message : "Registration failed.");
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <main className="mx-auto flex min-h-screen max-w-3xl items-center px-6 py-12
    md:px-10">
      <section className="glass-card w-full rounded-[36px] p-8 md:p-10">
        <p className="text-xs font-semibold uppercase tracking-[0.24em]
    text-[var(--accent-dark)]">
          Create account
        </p>
        <h1 className="mt-4 text-4xl font-semibold">Start reporting civic issues in
    minutes</h1>
        <form className="mt-8 grid gap-4" onSubmit={handleSubmit}>
          <input
            className="rounded-2xl border border-[var(--border)] bg-white/85 px-4 py-3
     outline-none"
            placeholder="Full name"
            value={name}
            onChange={(event) => setName(event.target.value)}
          />
          <input
            className="rounded-2xl border border-[var(--border)] bg-white/85 px-4 py-3
     outline-none"
            placeholder="Email address"
            type="email"
            value={email}
            onChange={(event) => setEmail(event.target.value)}
          />
          <input
            className="rounded-2xl border border-[var(--border)] bg-white/85 px-4 py-3
```

```
     outline-none"
            placeholder="Password"
            type="password"
            value={password}
            onChange={(event) => setPassword(event.target.value)}
          />
          <button
            className="rounded-full bg-[var(--accent)] px-5 py-3 font-semibold text-
    white transition hover:bg-[var(--accent-dark)] disabled:opacity-60"
            disabled={submitting}
            type="submit"
          >
            {submitting ? "Creating account..." : "Create account"}
          </button>
        </form>
        {message ? <p className="mt-4 text-sm text-[var(--ink-muted)]">{message}</p> :
     null}
        <p className="mt-5 text-sm text-[var(--ink-muted)]">
          Already registered?{" "}
          <Link href="/login" className="font-semibold text-[var(--foreground)]">
            Go to login
          </Link>
        </p>
      </section>
    </main>
  );
}
```

Web Dashboard Shell

File: web_app/src/components/DashboardShell.tsx

```
"use client";

import { useEffect, useState } from "react";

import ComplaintCard from "./Complaintcard";
import MapComplaint from "./MapComplaint";
import ReportPanel from "./ReportPanel";
import { type AdminOverview, type Complaint, demoOverview } from "../lib/demoData";
import { getAdminOverview } from "../services/adminService";
import { getComplaints } from "../services/complaintService";

export default function DashboardShell() {
  const [overview, setOverview] = useState<AdminOverview>(demoOverview);
  const [complaints, setComplaints] =
    useState<Complaint[]>(demoOverview.recentComplaints);

  const load = () =>
    Promise.all([getAdminOverview(), getComplaints()]).then(([nextOverview,
    nextComplaints]) => {
      setOverview(nextOverview);
      setComplaints(nextComplaints);
    });

  useEffect(() => {
    void load();
  }, []);

  const stats = [
    { label: "Open complaints", value: overview.totals.complaints },
    { label: "Pending", value: overview.totals.pending },
    { label: "In progress", value: overview.totals.inProgress },
    { label: "Resolved", value: overview.totals.resolved },
  ];

  return (
    <div className="grid gap-8">
      <section className="grid gap-4 md:grid-cols-4">
        {stats.map((stat) => (
          <div key={stat.label} className="glass-card rounded-[28px] p-5">
            <p className="text-sm text-[var(--ink-muted)]">{stat.label}</p>
            <p className="mt-3 text-3xl font-semibold">{stat.value}</p>
          </div>
        ))}
      </section>
      <section className="section-grid">
        <ReportPanel onCreated={load} />
        <MapComplaint complaints={complaints.slice(0, 3)} />
      </section>
      <section className="grid gap-5 md:grid-cols-2">
        {complaints.map((complaint) => (
          <ComplaintCard key={complaint.id} complaint={complaint} />
        ))}
      </section>
    </div>
  );
}
```

Web Report Panel

File: web_app/src/components/ReportPanel.tsx

```
"use client";

import { useState } from "react";

import { createComplaint } from "../services/complaintService";

type Props = {
  onCreated?: () => void;
};

export default function ReportPanel({ onCreated }: Props) {
  const [form, setForm] = useState({
    title: "",
    description: "",
    location: "",
  });
  const [submitting, setSubmitting] = useState(false);
  const [message, setMessage] = useState("");

  const updateField = (key: keyof typeof form, value: string) => {
    setForm((current) => ({ ...current, [key]: value }));
  };

  const handleSubmit = async (event: React.FormEvent<HTMLFormElement>) => {
    event.preventDefault();
    setSubmitting(true);
    setMessage("");

    try {
      await createComplaint(form);
      setMessage("Complaint submitted and routed for triage.");
      setForm({ title: "", description: "", location: "" });
      onCreated?.();
    } catch {
      setMessage("Unable to submit complaint right now.");
    } finally {
      setSubmitting(false);
    }
  };

  return (
    <section className="glass-card rounded-[32px] p-6">
      <p className="text-xs font-semibold uppercase tracking-[0.24em]
    text-[var(--accent-dark)]">
        Report an issue
      </p>
      <h2 className="mt-2 text-2xl font-semibold">Send a complaint into the response
    queue</h2>
      <form className="mt-6 grid gap-4" onSubmit={handleSubmit}>
        <input
          className="rounded-2xl border border-[var(--border)] bg-white/85 px-4 py-3
    outline-none"
          placeholder="Issue title"
          value={form.title}
          onChange={(event) => updateField("title", event.target.value)}
        />
        <textarea
          className="min-h-32 rounded-2xl border border-[var(--border)] bg-white/85
    px-4 py-3 outline-none"
          placeholder="Describe what happened"
          value={form.description}
          onChange={(event) => updateField("description", event.target.value)}
        />
        <input
          className="rounded-2xl border border-[var(--border)] bg-white/85 px-4 py-3
    outline-none"
          placeholder="Location"
          value={form.location}
          onChange={(event) => updateField("location", event.target.value)}
```

```
        />
        <button
          className="rounded-full bg-[var(--foreground)] px-5 py-3 text-sm font-
    semibold text-white transition hover:bg-[var(--accent-dark)] disabled:opacity-60"
          disabled={submitting}
          type="submit"
        >
          {submitting ? "Submitting..." : "Submit Complaint"}
        </button>
        {message ? <p className="text-sm text-[var(--ink-muted)]">{message}</p> :
    null}
      </form>
    </section>
  );
}
```

Web Complaint Card

File: web_app/src/components/Complaintcard.tsx

```
import type { Complaint } from "../lib/demoData";

type Props = {
  complaint: Complaint;
};

export default function ComplaintCard({ complaint }: Props) {
  return (
    <article className="glass-card rounded-[28px] p-5">
      <div className="mb-4 flex items-start justify-between gap-4">
        <div>
          <p className="text-xs font-semibold uppercase tracking-[0.24em]
    text-[var(--ink-muted)]">
            {complaint.category}
          </p>
          <h3 className="mt-2 text-xl font-semibold">{complaint.title}</h3>
        </div>
        <span className="rounded-full bg-[rgba(42,157,143,0.12)] px-3 py-1 text-xs
    font-semibold text-[var(--success)]">
          {complaint.status.replace("_", " ")}
        </span>
      </div>
      <p className="text-sm leading-6 text-[var(--ink-
    muted)]">{complaint.description}</p>
      <div className="mt-5 flex flex-wrap gap-3 text-sm">
        <span className="rounded-full bg-white/70 px-3
    py-1">{complaint.location}</span>
        <span className="rounded-full bg-[rgba(239,131,84,0.14)] px-3 py-1
    text-[var(--accent-dark)]">
          Priority {complaint.priority}
        </span>
        {complaint.department ? (
          <span className="rounded-full bg-[rgba(20,33,61,0.08)] px-3
    py-1">{complaint.department}</span>
        ) : null}
      </div>
      {complaint.suggestedAction ? (
        <p className="mt-4 text-sm leading-6 text-[var(--ink-muted)]">
          Suggested action: {complaint.suggestedAction}
        </p>
      ) : null}
    </article>
  );
}
```

Web Map Complaint

File: web_app/src/components/MapComplaint.tsx

```
import type { Complaint } from "../lib/demoData";

type Props = {
  complaints: Complaint[];
};

export default function MapComplaint({ complaints }: Props) {
  return (
    <div className="glass-card rounded-[32px] p-6">
      <div className="mb-5 flex items-center justify-between">
        <div>
          <p className="text-xs font-semibold uppercase tracking-[0.24em]
    text-[var(--ink-muted)]">
            Map Snapshot
          </p>
          <h2 className="mt-2 text-2xl font-semibold">Hotspots by locality</h2>
        </div>
        <span className="rounded-full bg-white/70 px-3 py-2 text-sm">
          {complaints.length} active markers
        </span>
      </div>
      <div className="relative overflow-hidden rounded-[28px] border
    border-[var(--border)] bg-[linear-gradient(140deg,#dbeafe,#fef3c7,#fde68a)] p-6">
        <div className="grid gap-4 md:grid-cols-3">
          {complaints.map((complaint, index) => (
            <div
              key={complaint.id}
              className="rounded-3xl bg-white/85 p-4 shadow-sm"
              style={{ transform: `translateY(${index % 2 === 0 ? "0px" : "10px"})` }}
            >
              <p className="text-xs font-semibold uppercase tracking-[0.22em]
    text-[var(--ink-muted)]">
                Zone {index + 1}
              </p>
              <h3 className="mt-2 font-semibold">{complaint.location}</h3>
              <p className="mt-2 text-sm text-[var(--ink-
    muted)]">{complaint.title}</p>
            </div>
          ))}
        </div>
      </div>
    </div>
  );
}
```

Web Sidebar

File: web_app/src/components/Sidebar.tsx

```
import Link from "next/link";

const links = [
  { href: "/dashboard", label: "Operations Overview" },
  { href: "/complaints", label: "All Complaints" },
  { href: "/admin/dashboard", label: "Admin Console" },
  { href: "/admin/analytics", label: "Analytics" },
];

export default function Sidebar() {
  return (
    <aside className="glass-card rounded-[32px] p-6">
      <p className="text-xs font-semibold uppercase tracking-[0.24em] text-[var(--ink-
    muted)]">
        Quick Navigation
      </p>
      <div className="mt-4 flex flex-col gap-3">
        {links.map((link) => (
          <Link
            key={link.href}
            href={link.href}
            className="rounded-2xl border border-[var(--border)] bg-white/60 px-4 py-3
     text-sm font-medium transition hover:-translate-y-0.5 hover:bg-white"
          >
            {link.label}
          </Link>
        ))}
      </div>
    </aside>
  );
}
```

Web Navbar

File: web_app/src/components/Navbar.tsx

```
import Link from "next/link";

const navLinks = [
  { href: "/dashboard", label: "Dashboard" },
  { href: "/complaints", label: "Complaints" },
  { href: "/admin/dashboard", label: "Admin" },
  { href: "/map", label: "Map" },
  { href: "/profile", label: "Profile" },
];

export default function Navbar() {
  return (
    <header className="sticky top-0 z-10 px-6 py-5 md:px-10">
      <div className="glass-card mx-auto flex max-w-6xl items-center justify-between
    rounded-full px-5 py-3">
        <Link href="/" className="text-lg font-semibold tracking-[0.18em] uppercase">
          UrbanEye
        </Link>
        <nav className="hidden gap-5 text-sm font-medium text-[var(--ink-muted)]
    md:flex">
          {navLinks.map((link) => (
            <Link key={link.href} href={link.href} className="transition
    hover:text-[var(--foreground)]">
              {link.label}
            </Link>
          ))}
        </nav>
        <Link
          href="/login"
          className="rounded-full bg-[var(--foreground)] px-4 py-2 text-sm font-medium
     text-white transition hover:bg-[var(--accent-dark)]"
        >
          Citizen Login
        </Link>
      </div>
    </header>
  );
}
```

Web Complaint Service

File: web_app/src/services/complaintService.ts

```
import { apiRequest } from "../lib/api";
import { demoComplaints, type Complaint } from "../lib/demoData";

export type ComplaintPayload = {
  title: string;
  description: string;
  location: string;
  latitude?: number;
  longitude?: number;
  image?: string;
};

function buildFallbackSocialPost(payload: ComplaintPayload) {
  return `Civic update: ${payload.title} at ${payload.location}.
    ${payload.description} #CityUpdate #CivicAction`;
}

export async function getComplaints() {
  return apiRequest<Complaint[]>("/complaints", {
    fallbackData: demoComplaints,
  });
}

export async function createComplaint(payload: ComplaintPayload) {
  return apiRequest<Complaint>("/complaints", {
    method: "POST",
    body: JSON.stringify(payload),
    fallbackData: {
      id: `demo-${Date.now()}`,
      title: payload.title,
      description: payload.description,
      location: payload.location,
      category: "General",
      priority: "MEDIUM",
      status: "PENDING",
      department: "Civic Response Cell",
      createdAt: new Date().toISOString(),
      socialPost: buildFallbackSocialPost(payload),
    },
  });
}
```

Web Admin Service

File: web_app/src/services/adminService.ts

```
import { apiRequest } from "../lib/api";
import { demoOverview, type AdminOverview, type Complaint, type User } from
    "../lib/demoData";

export async function getAdminOverview() {
  return apiRequest<AdminOverview>("/admin/overview", {
    fallbackData: demoOverview,
  });
}

export async function getAdminComplaints() {
  return apiRequest<Complaint[]>("/admin/complaints", {
    fallbackData: demoOverview.recentComplaints,
  });
}

export async function getAdminUsers() {
  return apiRequest<User[]>("/admin/users", {
    fallbackData: [
      { name: "Aarav Singh", email: "citizen@urbaneye.dev", role: "CITIZEN" },
      { name: "Meera Rao", email: "authority@urbaneye.dev", role: "AUTHORITY" },
      { name: "Sonal Verma", email: "admin@urbaneye.dev", role: "ADMIN" },
    ],
  });
}
```

Web Map Service

File: web_app/src/services/mapService.ts

```
import { apiRequest } from "../lib/api";
import { demoComplaints } from "../lib/demoData";

export async function getHotspots() {
  return apiRequest("/map/hotspots", {
    fallbackData: demoComplaints.map((item) => ({
      id: item.id,
      title: item.title,
      location: item.location,
      latitude: item.latitude,
      longitude: item.longitude,
      category: item.category,
      priority: item.priority,
      status: item.status,
    })),
  });
}
```

Web Auth Service

File: web_app/src/services/authService.ts

```
import { apiRequest } from "../lib/api";

export type AuthPayload = {
  name?: string;
  email: string;
  password: string;
};

export type AuthResponse = {
  user: {
    id: string;
    name: string;
    email: string;
    role: string;
    createdAt: string;
  };
};

function buildDemoUser(payload: AuthPayload) {
  return {
    id: `demo-${Date.now()}`,
    name: payload.name || "Demo Citizen",
    email: payload.email,
    role: "CITIZEN",
    createdAt: new Date().toISOString(),
  };
}

export async function loginUser(payload: AuthPayload) {
  return apiRequest<AuthResponse>("/auth/login", {
    method: "POST",
    body: JSON.stringify(payload),
    fallbackData: {
      user: buildDemoUser(payload),
    },
  });
}

export async function registerUser(payload: AuthPayload) {
  return apiRequest<AuthResponse>("/auth/register", {
    method: "POST",
    body: JSON.stringify(payload),
    fallbackData: {
      user: buildDemoUser(payload),
    },
  });
}
```

Mobile Navigator

File: mobile_app/src/navigation/AppNavigator.js

```
import React, { useContext } from "react";
import { createNativeStackNavigator } from "@react-navigation/native-stack";
import { ActivityIndicator, SafeAreaView, StyleSheet, Text, View } from "react-
    native";

import HomeScreen from "../screens/HomeScreen";
import ReportScreen from "../screens/ReportScreen";
import DashboardScreen from "../screens/DashbboardScreen";
import ComplaintDetailScreen from "../screens/ComplaintDetailScreen";
import MapScreen from "../screens/map";
import ProfileScreen from "../screens/ProfileScreen";

import LoginScreen from "../screens/LoginScreen";
import RegisterScreen from "../screens/RegisterScreen";

import { AuthContext } from "../context/AuthContext";

const Stack = createNativeStackNavigator();

export default function AppNavigator() {
  const { isHydrated, user } = useContext(AuthContext);

  if (!isHydrated) {
    return (
      <SafeAreaView style={styles.safeArea}>
        <View style={styles.loadingCard}>
          <ActivityIndicator color="#ef8354" size="large" />
          <Text style={styles.loadingTitle}>Loading your civic workspace</Text>
          <Text style={styles.loadingBody}>Restoring profile, complaint tracker, and
    saved X account.</Text>
        </View>
      </SafeAreaView>
    );
  }

  return (
    <Stack.Navigator
      screenOptions={{
        headerBackTitle: "Back",
        headerStyle: {
          backgroundColor: "#f4efe6",
        },
        headerShadowVisible: false,
        headerTintColor: "#14213d",
        headerTitleStyle: {
          fontWeight: "700",
        },
      }}
    >
      {user ? (
        <>
          <Stack.Screen name="Home" component={HomeScreen} options={{ title:
    "UrbanEye" }} />
          <Stack.Screen name="Report" component={ReportScreen} options={{ title:
    "Report Issue" }} />
          <Stack.Screen name="Dashboard" component={DashboardScreen} options={{ title:
     "Dashboard" }} />
          <Stack.Screen name="Detail" component={ComplaintDetailScreen} options={{
    title: "Complaint Detail" }} />
          <Stack.Screen name="Map" component={MapScreen} options={{ title: "Hotspot
    Map" }} />
          <Stack.Screen name="Profile" component={ProfileScreen} options={{ title:
    "Profile" }} />
        </>
      ) : (
        <>
          <Stack.Screen name="Login" component={LoginScreen} options={{ headerShown:
    false }} />
          <Stack.Screen name="Register" component={RegisterScreen} options={{
```

```
    headerShown: false }} />
        </>
      )}
    </Stack.Navigator>
  );
}

const styles = StyleSheet.create({
  safeArea: {
    flex: 1,
    backgroundColor: "#f4efe6",
    justifyContent: "center",
    padding: 24,
  },
  loadingCard: {
    backgroundColor: "#14213d",
    borderRadius: 28,
    padding: 28,
    alignItems: "center",
  },
  loadingTitle: {
    marginTop: 18,
    color: "#fff",
    fontSize: 22,
    fontWeight: "800",
    textAlign: "center",
  },
  loadingBody: {
    marginTop: 10,
    color: "#dbe2ef",
    lineHeight: 22,
    textAlign: "center",
  },
});
```

Mobile Auth Context

File: mobile_app/src/context/AuthContext.js

```
import AsyncStorage from "@react-native-async-storage/async-storage";
import React, { createContext, useEffect, useState } from "react";

export const AuthContext = createContext();

const USER_STORAGE_KEY = "urbaneye-user";

export default function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [isHydrated, setIsHydrated] = useState(false);

  useEffect(() => {
    let isMounted = true;

    const hydrateUser = async () => {
      try {
        const storedUser = await AsyncStorage.getItem(USER_STORAGE_KEY);

        if (storedUser && isMounted) {
          setUser(JSON.parse(storedUser));
        }
      } catch (error) {
        if (isMounted) {
          setUser(null);
        }
      } finally {
        if (isMounted) {
          setIsHydrated(true);
        }
      }
    };

    hydrateUser();

    return () => {
      isMounted = false;
    };
  }, []);

  useEffect(() => {
    if (!isHydrated) {
      return;
    }

    const persistUser = async () => {
      if (user) {
        await AsyncStorage.setItem(USER_STORAGE_KEY, JSON.stringify(user));
        return;
      }

      await AsyncStorage.removeItem(USER_STORAGE_KEY);
    };

    persistUser();
  }, [isHydrated, user]);

  const updateUser = (updates) => {
    setUser((currentUser) => {
      if (!currentUser) {
        return currentUser;
      }

      const nextUpdates =
        typeof updates === "function" ? updates(currentUser) : updates;

      return {
        ...currentUser,
        ...nextUpdates,
      };
```

```
    });
  };

  return (
    <AuthContext.Provider value={{ isHydrated, updateUser, user, setUser }}>
      {children}
    </AuthContext.Provider>
  );
}
```

Mobile Complaint Service

File: mobile_app/src/services/complaintService.js

```
import AsyncStorage from "@react-native-async-storage/async-storage";

import API from "./api";

const LOCAL_COMPLAINTS_KEY = "urbaneye-local-complaints";

function buildFallbackSocialPost(payload) {
  return `Civic update: ${payload.title} at ${payload.location}.
    ${payload.description} #CityUpdate #CivicAction`;
}

async function loadLocalComplaints() {
  try {
    const rawComplaints = await AsyncStorage.getItem(LOCAL_COMPLAINTS_KEY);
    return rawComplaints ? JSON.parse(rawComplaints) : [];
  } catch (error) {
    return [];
  }
}

async function saveLocalComplaints(complaints) {
  try {
    await AsyncStorage.setItem(LOCAL_COMPLAINTS_KEY, JSON.stringify(complaints));
  } catch (error) {
    // Keep the app responsive even if local persistence fails.
  }
}

function mergeComplaints(primaryComplaints, secondaryComplaints = []) {
  const seenIds = new Set();
  const combined = [...primaryComplaints, ...secondaryComplaints].filter((complaint)
    => {
    if (seenIds.has(complaint.id)) {
      return false;
    }

    seenIds.add(complaint.id);
    return true;
  });

  return combined.sort((left, right) => new Date(right.createdAt || 0) - new
    Date(left.createdAt || 0));
}

async function persistComplaint(complaint) {
  const currentComplaints = await loadLocalComplaints();
  const updatedComplaints = mergeComplaints([complaint], currentComplaints);
  await saveLocalComplaints(updatedComplaints);
}

const demoComplaints = [
  {
    id: "cmp-1001",
    title: "Overflowing roadside garbage",
    description: "Garbage bins near Sector 8 market have not been cleared for three
    days.",
    location: "Sector 8 Market, Patna",
    category: "Sanitation",
    priority: "HIGH",
    status: "IN_PROGRESS",
    department: "Sanitation Department",
    suggestedAction: "Dispatch sanitation pickup team within 12 hours.",
    socialPost: "Ticket escalated for sanitation crew update.",
    latitude: 25.6175,
    longitude: 85.1452,
  },
  {
    id: "cmp-1002",
    title: "Streetlight outage",
```

```
    description: "Two consecutive streetlights are out near the school crossing.",
    location: "Boring Road Crossing, Patna",
    category: "Electricity",
    priority: "MEDIUM",
    status: "PENDING",
    department: "Electricity Department",
    suggestedAction: "Assign to night maintenance team for inspection.",
    socialPost: "",
    latitude: 25.6128,
    longitude: 85.1178,
  },
  {
    id: "cmp-1003",
    title: "Water leakage near bus stop",
    description: "Continuous water leakage is flooding the footpath near the bus
    stop.",
    location: "Kankarbagh Main Road, Patna",
    category: "Water",
    priority: "HIGH",
    status: "PENDING",
    department: "Water Department",
    suggestedAction: "Inspect pipeline pressure and send repair crew.",
    socialPost: "",
    latitude: 25.5944,
    longitude: 85.1612,
  },
];

export const getComplaints = async () => {
  const localComplaints = await loadLocalComplaints();

  try {
    const res = await API.get("/complaints");
    return mergeComplaints(res.data, localComplaints);
  } catch (error) {
    return mergeComplaints(localComplaints, demoComplaints);
  }
};

export const createComplaint = async (payload) => {
  try {
    const res = await API.post("/complaints", payload);
    await persistComplaint(res.data);
    return res.data;
  } catch (error) {
    const complaint = {
      id: `demo-${Date.now()}`,
      title: payload.title,
      description: payload.description,
      location: payload.location,
      category: "General",
      priority: "MEDIUM",
      status: "PENDING",
      department: "Civic Response Cell",
      suggestedAction: "Validate details and route to the appropriate department.",
      socialPost: buildFallbackSocialPost(payload),
      createdAt: new Date().toISOString(),
    };

    await persistComplaint(complaint);
    return complaint;
  }
};
```

Mobile API Service

File: mobile_app/src/services/api.js

```
import axios from "axios";

const API = axios.create({
    baseURL: process.env.EXPO_PUBLIC_API_URL || "http://192.168.138.51:5000/api",
});

export default API;
```

Mobile Auth Service

File: mobile_app/src/services/authService.js

```
import API from "./api";

function buildDemoUser(data) {
  return {
    id: `demo-${Date.now()}`,
    name: data.name || "Demo Citizen",
    email: data.email,
    role: "CITIZEN",
    createdAt: new Date().toISOString(),
  };
}

export const loginUser = async (data) => {
  try {
    const res = await API.post("/auth/login", data);
    return res.data;
  } catch (error) {
    if (!error.response) {
      return { user: buildDemoUser(data) };
    }

    throw error;
  }
};

export const registerUser = async (data) => {
  try {
    const res = await API.post("/auth/register", data);
    return res.data;
  } catch (error) {
    if (!error.response) {
      return { user: buildDemoUser(data) };
    }

    throw error;
  }
};
```